

Cartridge & Cloud - Unity Project Setup Guide

Configuración reproducible y estado validado del proyecto

VRM Games / Blas Luis Rocha González

22/06/2026

Índice

0. Configuración validada en Sprint 0	2
0.1 Paquetes directos conservados	2
0.2 Validación de restauración	2
1. Objetivo	2
2. Decisiones iniciales	2
3. Creación	2
4. Paquetes	2
5. Carpetas	3
6. Assembly Definitions	3
7. Escenas	3
8. Project Settings	3
9. URP	3
10. Git	3
11. CI/build local	3
12. Checklist Sprint 0	4

Proyecto: Cartridge & Cloud

Título técnico provisional: Cartridge & Cloud

Plataforma inicial: PC / Steam

Motor validado: Unity 6.3 LTS 6000.3.18f1 / C# / URP 17.3.0

Versión del documento: v0.4

Estado: Baseline documental posterior al cierre de Sprint 0 **Fuente conceptual:** Enfoque_v0.6.md

Regla de interpretación v0.4: las afirmaciones de implementación se limitan a la base técnica validada durante Sprint 0. Los sistemas de gameplay y el vertical slice continúan como especificación hasta que exista evidencia posterior.

0. Configuración validada en Sprint 0

Parámetro	Valor
Nombre técnico	<code>CartridgeAndCloud</code>
Company / Product	VRM Games / Cartridge & Cloud
Unity	6000.3.18f1
Template / Render	3D URP / URP 17.3.0
Color space	Linear
Versión	0.0.1
Application Identifier	<code>com.vrmgames.cartridgeandcloud</code>
Plataforma	Windows Intel 64-bit
Backend de desarrollo	Mono
Build compression	LZ4
Player Log	Activado

0.1 Paquetes directos conservados

AI Navigation, Visual Studio Editor, Input System, Localization, Universal RP, Test Framework y Unity UI.

Se retiraron Rider Editor, Collab Proxy, Multiplayer Center, Visual Scripting y Timeline al no formar parte de la arquitectura activa.

0.2 Validación de restauración

Una restauración correcta debe abrir sin errores, resolver paquetes, mostrar las seis asmdefs, conservar las cuatro escenas, ejecutar 9/9 tests y generar un Player Windows fuera del Editor.

1. Objetivo

Documentar la base reproducible ya creada para PC/Steam y definir cómo verificarla en una máquina nueva.

2. Decisiones iniciales

- Unity 6 LTS, versión exacta fijada en `ProjectVersion.txt`.
- Plantilla 3D URP.
- Windows x64.
- Input System.
- TextMeshPro y uGUI.
- AI Navigation.
- Unity Test Framework.
- Localization cuando empiece UI estable.
- Git LFS solo para binarios grandes justificados.

3. Creación

1. Crear proyecto `CartridgeAndCloud` en Unity Hub.
2. Company Name `VRM Games`.
3. Product Name provisional `Cartridge & Cloud`.
4. Configurar Visible Meta Files y Force Text.
5. Añadir `.gitignore` de Unity.
6. Crear repositorio y rama `main` protegida conceptualmente.
7. Hacer build Windows vacío y registrar evidencia.

4. Paquetes

Paquete	Momento
Input System	Sprint 0
AI Navigation	Sprint 0
TextMeshPro	Sprint 0
Test Framework	Sprint 0
Cinemachine	Solo si el spike demuestra valor
Localization	Antes de textos masivos
Addressables	Cuando el volumen de contenido lo justifique
Steamworks	No antes de una build representativa

5. Carpetas

```
Assets/_Project/
  Art/Characters/Environment/Props/Products
  Audio/Music/SFX/Mixers
  Content/Definitions/Balance/Localization
  Code/Domain/Application/Infrastructure/Presentation/Editor/Tests
  Prefabs/Furniture/Characters/UI/Systems
  Scenes/Bootstrap/MainMenu/Store/TestLab
  Settings
  UI/Fonts/Icons/Sprites
```

6. Assembly Definitions

```
VRMGames.CartridgeAndCloud.Domain, .Application, .Infrastructure, .Presentation, .Editor, .Tests.EditMode,
.Tests.PlayMode.
```

7. Escenas

Bootstrap carga servicios; MainMenu no mantiene simulación; Store contiene mundo y composición; TestLab permite pruebas aisladas. Evitar managers duplicados por escena.

8. Project Settings

- Color space Linear.
- Input Handling: Input System Package.
- Scripting backend Mono para desarrollo; IL2CPP se valida antes de release.
- API Compatibility según LTS.
- Run in Background configurable.
- Resolution 1920x1080 por defecto.
- VSync y frame cap configurables.

9. URP

Calidad Low/Medium/High, sombras contenidas, postprocesado mínimo, MSAA según perfil y luces baked/mixtas. No introducir efectos caros antes de medir.

10. Git

No versionar Library, Temp, Obj, Logs, UserSettings ni Builds. Versionar Packages, ProjectSettings, Assets y documentación. Commits atómicos con Conventional Commits adaptado.

11. CI/build local

Al menos un script de build reproducible antes del vertical slice. El pipeline debe validar compilación, tests y datos antes de generar artefacto.

12. Checklist Sprint 0

- ☐ Proyecto abre sin warnings críticos.
- ☐ Paquetes resueltos.
- ☐ Build Windows ejecuta.
- ☐ Tests de ejemplo pasan.
- ☐ Escenas creadas.
- ☐ Git limpio.
- ☐ README y versión actualizados.
- ☐ Estructura legal y de documentación creada.